Modular

MOJO WORKSHOP · LLVM DEVELOPERS' MEETING 2026

Mojo roadmap

Language · Standard library · SDK · Community



Mojo Roadmap for 2026-2027

- Language and standard library
- Compiler improvements
- SDK / toolchain
- Grow user community

mojolang.org/docs/roadmap

Mojo's focus now: kernel programming

- Mojo was built for kernel programming: a small pool of people with specific needs
- Kernels are hyper-parametric and extremely performance-focused
- Emphasis was on generics, comptime evaluation, type-system features, MLIR integration

These continue to be important. What we plan to add to Mojo, to make it a better kernel language:

- Enumerations & pattern matching
- More compile-time metaprogramming, making it easier to add “eDSL” features
- Syntax for hardware concepts e.g. address spaces
- Speed up the compilation process (forever project)



New goal in 2027: CPU programming

- Mojo wants to jump to a bigger pool: developers who write performant code, but not necessarily GPU AI kernels
- New use case: use Mojo in the Modular AI inference server

Those developers have different needs, more like Rust:

- Async programming (coroutines, async/await)
- Dynamic dispatch (dyn traits, existentials)
- Threading libraries and concurrency model
- Networking libraries
- Better interoperability with C (and other languages)

They also expect more from the SDK & toolchain



Compiler improvements

2026:

- Compiler speedup
- Make GPU and NPU back-ends source available
- Pluggable MLIR and LLVM back-ends (gated by switching to dynamic linking of LLVM)
- Enable WASM back-end

2027

- More vendor back-ends!
- Define undefined behaviors
- FP consistency
- Design Mojo CPU benchmarks and start working on performance
- Gradually move optimization passes from LLVM to MLIR
- Incremental compilation

SDK / Toolchain

Programming language has to have a suite of tools that cover the entire development cycle
What we plan to do, roughly in this order:

- Automatic Mojo / C interop (bindgen)
- Debugger (make LLDB work)
- Testing framework
- Improve Formatter and LSP
- Fix REPL and Jupyter integration
- Improve Mojo docgen

What tools does a modern language need? Do debuggers still matter in the AI era?
We would love to hear your thoughts.

Grow the Mojo user community

- Package manager and registry
- Mojo build system
- Windows support

Package manager and registry

- No thriving community until it is easy to create, share, and use Mojo libraries
- We will build on pixi, improved to install GPU-versioned binaries
- Then build a Mojo front-end for it, so the experience is similar to Rust's cargo

```
# Mojo library developer creates a git repo
gh init cool_lib
# work happens
...
PIXI_TOKEN=... # get this from Modular
pixi publish
# registers the cool-lib with github/modular-community
# new releases are pushed and updated automatically
```

```
# Mojo application developer:
pixi add cool_lib # Pixi + build system figures
everything out from here
```

- Then build a package registry similar to crates.io

Mojo build system

- How to build programs written in Mojo, and programs where Mojo is part of the solution?
- We plan to use Bazel, with rules and wrappers to make it feel less difficult to use
- Most of the build system will be autogenerated from Pixi config
- Bazel stays available for custom configuration



Windows support

- How many people really use Windows? It depends on whom Mojo serves
- AI kernel engineers: all on Linux
- Performant server-side code: likely all on Linux
- Python-centric data science: perhaps half on Windows? Of them, half use WSL?
- No specific plan yet. We will make some moves early in 2027

Putting it all together on the timeline

	Q4 2026	Q1 2027	Q2 2027	Q3 2027 AND BEYOND
Language	• Pattern matching and enumerations • Preview of async/await • Some networking support	• Async/await • Threading and networking libraries	• Existentials, runtime dispatch • Hygienic macros	
Compiler	• Pluggable back-ends • GPU and NPU backends source-available • Compiler speedup	• Mojo CPU benchmarks • Undefined behavior	• MLIR optimization passes • Incremental compilation	
SDK / Toolchain	• Package manager • Dynamically link LLVM across compiler and tools	• Mojo build system • Better Mojo / C interop (bindgen) • Debugger	• LSP and formatter improvements • Testing framework	• REPL • Jupyter • Windows? • Arduino?

Modular

THANK YOU

Questions?

BY THEWAY, WE ARE HIRING!

Front-end and back-end compiler engineers, tooling, debugger, libraries, and hardware bring-up. We have both junior and senior roles.

modular.com/careers